LACNet Smart Contract Deployment Guide - Manglar Azul MRV

LACNET SMART CONTRACT DEPLOYMENT GUIDE

Manglar Azul MRV - Technical Implementation

Section 8: Deploy Smart Contract - Step by Step Guide

Based on the LACNet deployment model and the hackathon sample structure, this guide explains how to deploy the ManglarCarbonCredit smart contract for blue-carbon tokenization.

1. DEVELOPMENT ENVIRONMENT SETUP

Required components

Install:

- Node.js 20+ or 18 LTS.
- npm.
- Git.
- A LACNet-compatible wallet private key for testnet deployment.
- Access to the LACNet RPC endpoint provided by the hackathon.

Recommended verification commands:

```
node --version
npm --version
git --version
```

Project dependencies

From the repository root:

```
cd manglar-token
npm install
```

Main development packages:

```
hardhat
@nomicfoundation/hardhat-toolbox
dotenv
ethers
```

LACNet gas model configuration

The current project uses a zero-gas-price EVM configuration compatible with the hackathon guidance:

```
require("@nomicfoundation/hardhat-toolbox");
require("dotenv").config();

const PRIVATE_KEY = process.env.PRIVATE_KEY || "";
const LACNET_RPC_URL = process.env.LACNET_RPC_URL || "http://35.193.217.67";
const LACNET_CHAIN_ID = Number(process.env.LACNET_CHAIN_ID || 648529);

module.exports = {
  solidity: {
    version: "0.8.24",
    settings: {
      optimizer: {
        enabled: true,
        runs: 200
      }
    }
  },
  networks: {
    hardhat: {
      chainId: 31337
    },
    lacnet: {
      url: LACNET_RPC_URL,
      chainId: LACNET_CHAIN_ID,
      accounts: PRIVATE_KEY ? [PRIVATE_KEY] : [],
      gasPrice: 0
    }
  }
};
```

If the active LACNet node requires the official gas model provider, add it as an optional integration:

```
npm install @lacchain/gas-model-provider
```

Then adapt the provider settings according to the current LACNet documentation and node credentials.

2. PROJECT STRUCTURE INITIALIZATION

The repository is already initialized with the following structure:

```
manglar-token/
|-- contracts/
|   `-- ManglarCarbonCredit.sol
|-- scripts/
|   |-- deploy.js
|   |-- demo-flow.js
|   `-- export_pdfs.py
|-- mrv/
|   |-- mrv_pipeline.py
|   `-- output/
|       |-- mrv_report.json
|       `-- mrv_report.txt
|-- data/
|   |-- mrv_observations.csv
|   `-- project_metadata.json
|-- demo/
|   |-- index.html
|   |-- styles.css
|   `-- app.js
|-- docs/
|-- test/
|-- pitch/
|-- entregables_pdf/
|-- hardhat.config.js
|-- package.json
```

```
`-- README.md
```

3. ENVIRONMENT VARIABLES

Create a local `.env` file from the example:

```
cp .env.example .env
```

Windows PowerShell alternative:

```
Copy-Item .env.example .env
```

Example `.env`:

```
PRIVATE_KEY=replace_with_testnet_private_key
LACNET_RPC_URL=http://35.193.217.67
LACNET_CHAIN_ID=648529
PROJECT_STEWARD=0x0000000000000000000000000000000000000000
INITIAL_BUYER=0x0000000000000000000000000000000000000000
```

Security note:

- Never commit `.env`.
- Use only a testnet wallet.
- Do not use a wallet with mainnet funds for hackathon deployment.

4. SMART CONTRACT IMPLEMENTATION

4.1 Core contract

Main file:

```
contracts/ManglarCarbonCredit.sol
```

The contract defines:

- `Project`: registered blue-carbon project metadata.
- `Batch`: tokenized carbon credit lot with vintage and MRV hash.
- `Retirement`: public retirement receipt.
- `oracles`: accounts allowed to update MRV evidence.
- `certifiers`: accounts allowed to issue verified batches.

4.2 Main functions

```
registerProject(name, location, steward, methodology, metadataURI)
issueBatch(projectId, to, amount, vintage, mrvHash, evidenceURI, data)
updateMrvEvidence(batchId, newMrvHash, newEvidenceURI)
transfer(to, batchId, amount)
safeTransferFrom(from, to, id, amount, data)
retire(batchId, amount, beneficiary, reason, certificateHash)
availableToRetire(batchId)
```

4.3 Token model

```
1 MACC = 1 verified tonne CO2e
```

Each token ID represents one MRV batch, not a generic fungible pool. This avoids mixing projects, vintages or evidence packages.

4.4 Audit events

Important events:

```
ProjectRegistered
BatchIssued
MrvEvidenceUpdated
TransferSingle
CreditRetired
URI
```

These events create the public audit trail from project registration to retirement.

5. MRV PIPELINE BEFORE DEPLOYMENT

Before issuing a batch, generate the MRV report:

```
python mrv/mrv_pipeline.py
```

Expected output:

```
Issueable credits: 2731 tCO2e
MRV hash: 0x3cbcd3d5dc5901126ac241f1ed3001f0264beebca291d27549e5a8a42cc8e1ba
```

Generated files:

```
mrv/output/mrv_report.json
mrv/output/mrv_report.txt
```

The deployment script reads `mrv/output/mrv_report.json` and uses the `issueable_tco2e` and `mrv_hash` values.

6. DEPLOYMENT SCRIPT

Main deployment file:

```
scripts/deploy.js
```

Deployment process:

1. Load the MRV report.
2. Deploy `ManglarCarbonCredit`.
3. Assign deployer as oracle and certifier.
4. Register the Manglar Azul Tumbes project.
5. Issue the first MRV-backed batch to the configured buyer.

Important deployment output:

```
ManglarCarbonCredit deployed to: <contract_address>
Project registered: 1
Batch issued: 1
Initial buyer: <wallet_address>
Issued tonnes CO2e: 2731
MRV hash: 0x3cbcd3d5dc5901126ac241f1ed3001f0264beebca291d27549e5a8a42cc8e1ba
```

7. DEPLOYMENT AND TESTING COMMANDS

7.1 Full local validation

```
npm install
python mrv/mrv_pipeline.py
npm run compile
npm test
node scripts/demo-flow.js
```

Windows PowerShell alternative:

```
npm.cmd install
python mrv/mrv_pipeline.py
npm.cmd run compile
npm.cmd test
node scripts\demo-flow.js
```

7.2 Deploy to local Hardhat network

Terminal 1:

```
npx hardhat node
```

Terminal 2:

```
npm run deploy:local
```

7.3 Deploy to LACNet testnet

Confirm `.env` first, then run:

```
python mrv/mrv_pipeline.py
npm run compile
npm run deploy:lacnet
```

PowerShell:

```
python mrv/mrv_pipeline.py
npm.cmd run compile
npm.cmd run deploy:lacnet
```

8. BASIC WEB INTERFACE FOR CONTRACT INTERACTION

The prototype includes a local functional demo:

```
demo/index.html
```

Demo actions:

1. Register project.
2. Issue MRV-backed batch.
3. Retire 25 tCO2e.
4. Display the public audit log and certificate hash.

Current demo mode is local/off-chain for presentation purposes. Future wallet integration should connect the UI to the deployed contract using `ethers.js`.

9. LACNET-SPECIFIC CONFIGURATION NOTES

RPC endpoint

Default hackathon RPC in `.env.example`:

```
LACNET_RPC_URL=http://35.193.217.67
```

If organizers provide a different writer RPC, replace the value in `.env`.

Chain ID

Default value:

```
LACNET_CHAIN_ID=648529
```

If the current testnet uses another chain ID, update `.env` before deployment.

Gas

The guide uses:

```
gasPrice: 0
```

If transactions fail due to provider-specific gas rules, use the latest LACNet gas model provider instructions and re-run deployment.

10. TROUBLESHOOTING GUIDE

Problem: Hardhat cannot create AppData folders on Windows

Use local Hardhat folders:

```
$root=(Resolve-Path '.').Path
$env:APPDATA="$root\.hardhat-global\Roaming"
$env:LOCALAPPDATA="$root\.hardhat-global\Local"
npm.cmd test
```

Problem: missing MRV report

Error:

```
Run `python mrv/mrv_pipeline.py` before deployment
```

Fix:

```
python mrv/mrv_pipeline.py
```

Problem: invalid private key

Check `.env`:

```
PRIVATE_KEY=0x...
```

Use a full testnet private key and never commit it.

Problem: network connection fails

Check:

```
curl <LACNET_RPC_URL>
```

Or verify that the hackathon RPC endpoint is still active.

Problem: transactions fail on LACNet gas model

Actions:

1. Confirm `gasPrice: 0`.
2. Confirm chain ID and RPC URL.
3. Check whether the official `@lacchain/gas-model-provider` is required by the active testnet.
4. Re-run deployment after updating configuration.

11. PRIORITY IMPLEMENTATION STRATEGY

Stage 1: Basic deployment

1. Compile contract.
2. Run tests.
3. Deploy to local Hardhat network.
4. Confirm project registration and batch issuance.

Stage 2: LACNet testnet deployment

1. Configure `.env`.
2. Generate MRV hash.
3. Deploy `ManglarCarbonCredit`.

4. Save address and transaction hashes.

5. Add explorer links to final submission.

Stage 3: Evidence storage

1. Upload MRV JSON and project metadata to IPFS/Filecoin.
2. Replace placeholder `ipfs://` URIs with real content identifiers.
3. Update `evidenceURI` on-chain if needed.

Stage 4: Wallet UI

1. Add wallet connection.
2. Read project/batch state from contract.
3. Execute retirement transactions from the UI.
4. Generate PDF retirement certificates.

12. NEXT STEPS

Immediate:

- Deploy to the current LACNet testnet once final RPC and wallet credentials are available.
- Save deployed address in a `deployments/lacnet-testnet.json` file.
- Record contract address and transaction hashes in the final Drive submission.

Short term:

- Replace illustrative MRV data with verifier-approved project evidence.
- Publish evidence and metadata on decentralized storage.

Medium term:

- Add verifier signatures and role governance.
- Build wallet-connected UI.

Long term:

- Pilot with one mangrove project and one institutional buyer.
- Expand to additional blue-carbon sites in Latin America.